HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

Prof. Yann Thoma

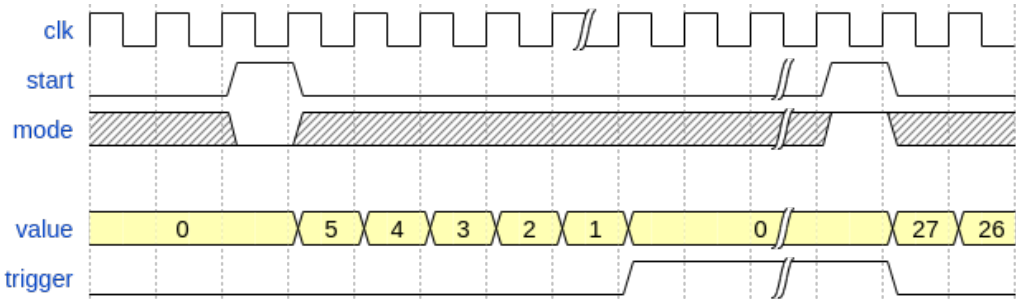
Formal verification
Verification of a timer
CERN training - November 2026

Context

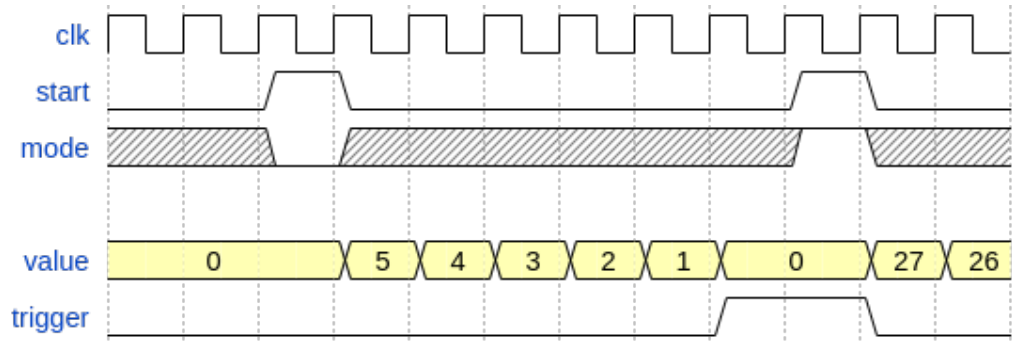
We are interested in verifying the correctness of a timer having the following interface :

```
entity timer is
port(
  clk_i     : in  std_logic;
  rst_i     : in  std_logic;
  start_i   : in  std_logic;
  mode_i    : in  std_logic;
  value_o   : out std_logic_vector(7 downto 0);
  trigger_o : out std_logic
);
end timer;
```

The timer starts when `start_i` is asserted for one clock cycle. At that clock edge, `mode_i` selects the duration : `'0'` selects 6 cycles and `'1'` selects 28 cycles. The timer then counts down to zero. When the countdown completes, `trigger_o` is asserted and remains asserted until the timer is restarted by asserting `start_i` again.
The current value of the counter is also available as the `value_o` output.
The specifications of the timer state the start input should not be reactivated before the timer is triggered.
The following waveform illustrates an example of how it works. The grayed-out mode indicates that the current value is not important.



And the next one the minimum time between two activations of `start` :



Propose a set of assertions (and maybe assumptions) that allow you to use formal verification to prove the correct behavior of this timer. Make a good use of `assume` and `assert` to formally verify this timer.

The timer has a fault injection capability that is able to generate errors, thanks to the `ERRNO` generic parameter.
The following `ERRNO` values are used to test the assertions :

- 0 : valid behavior
- in $[1, 5]$: wrong behavior

The provided SBY script tests `ERRNO` value 0 as valid configurations, and values from 1 to 5 as errors for which it is expected to fail. You can have a look at the `timer.sby` file to see how to launch verification with different engines `ERRNO` values.